POLYTECH NICE SOPHIA

Parcours des écoles d'ingénieurs Polytech (MAM3)

Introduction à la Programmation et à l'Algorithmique

Présentation : **D. Lopez-Pacheco, Hélène Renard**

Auteurs, D. LP, H.R., et al



Pour dynamiser et rendre interactif un cours en Cours/TD d'IPA …

Les conditions et les tests

Simples, multiples, ou « à la carte »

Le test simple : if



- ✓ On appelle traitement conditionnel une portion de code qui n'est pas exécutée systématiquement;
- ✓ Syntaxe du traitement conditionnel :

```
if (condition)
{
        instructions;
}
```

- ✓ Notez bien qu'il n'y a pas de point-virgule après la parenthèse du `if`.

Le test simple : if … else

✓ Syntaxe du traitement conditionnel étendu :

```
if (condition)
{
        instructions;
}
else
{
        instructions2;
}
```

✓ Lorsqu'**une seule instruction** d'un bloc `if` doit être exécutée, les accolades ne sont plus nécessaires.

Les différents tests possibles

Les opérateurs de comparaison en Java		
Opérateur	**Signification**	**Exemple**
`==`	égal à	`if (i == 5)`
`!=`	différent de	`if (i != 5)`
`<`	inférieur à	`if (i < 5)`
`<=`	inférieur ou égal à	`if (i <= 5)`
`>`	supérieur à	`if (i > 5)`
`>=`	supérieur ou égal à	`if (i >= 5)`



Je jongle avec les différents tests …
Et vous ?

```
class test {
        public static void main(String[] args) {
                int i;
                i=8;
                if (i == 10) { System.out.println("i égal à 10"); }
                else { System.out.println("i pas égal à 10"); }

                if (i != 10) { System.out.println("i différent de 10"); }
                else { System.out.println("i pas différent de 10"); }

                if (i < 10) { System.out.println("i inférieur à 10"); }
                else { System.out.println("i pas inférieur à 10"); }

                if (i <= 10) { System.out.println("i inf. ou égal à 10"); }
                else { System.out.println("i pas inf. ou égal à 10"); }

                if (i > 10) { System.out.println("i supérieur à 10"); }
                else { System.out.println("i pas supérieur à 10"); }

                if (i >= 10) { System.out.println("i sup. ou égal à 10"); }
                else { System.out.println("i pas sup. ou égal à 10"); }
        }
}
```

Utilisation des opérateurs logiques dans les conditions de test

Les opérateurs logiques en Java		
Opérateur	**Signification**	**Exemple**
`&&`	`ET logique`	`if ((i==5) && (j==8))`
`||`	`OU logique`	`if ((i>10) || (i<2))`
`!`	`NON logique`	`if !(i==9)`

Rappel des tables de vérité des opérateurs logiques de base :

Opérateur ET		
a	b	***a && b***
faux	faux	***FAUX***
faux	Vrai	***FAUX***
vrai	Faux	***FAUX***
vrai	vrai	***VRAI***

Opérateur OU		
a	b	***a || b***
faux	faux	***FAUX***
faux	vrai	***VRAI***
vrai	faux	***VRAI***
vrai	vrai	***VRAI***

Opérateur NON	
a	!a
faux	***VRAI***
vrai	***FAUX***

Propriétés logiques des conditions



- ✓ il y a souvent plusieurs manière d'écrire ce test.
- ✓ Exemple :

```
/* teste la différence : */
if (a != 5)
        {
                System.out.println("a est différent de 5");
        }
/* teste la non égalité : */
if !(a == 5)
        {
                System.out.println("a n'est pas égal à 5");
        }
```

Conditions équivalentes

✓ Le tableau suivant montre des exemples de condition égales deux à deux.

Conditions équivalentes	
La condition	**est strictement équivalente à**
`a==5`	`!(a!=5)`
`a!=5`	`!(a==5)`
`(a!=5) && (b!=8)`	`!((a==5) || (b==8))`
`(a!=5) || (b!=8)`	`!((a==5) && (b==8))`
`!((a!=5) && (b!=8))`	`(a==5) || (b==8)`
`!((a!=5) || (b!=8))`	`(a==5) && (b==8)`

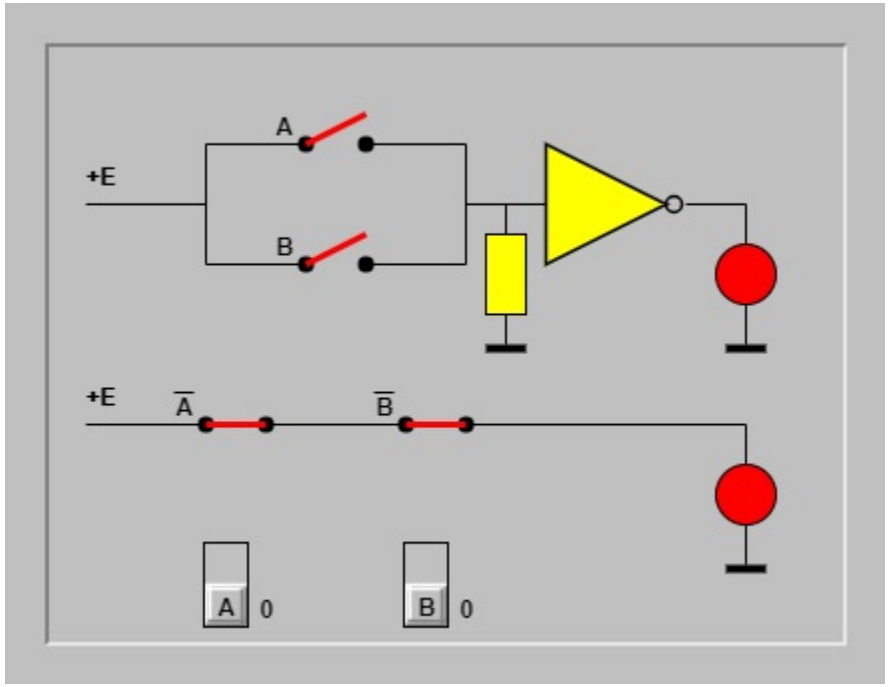


Le théorème de ***De Morgan*** *permet d'expliquer les 4 dernières lignes*

http://ressources.univ-lemans.fr/AccesLibre/UM/Pedago/physique/02/electro/morgan.html

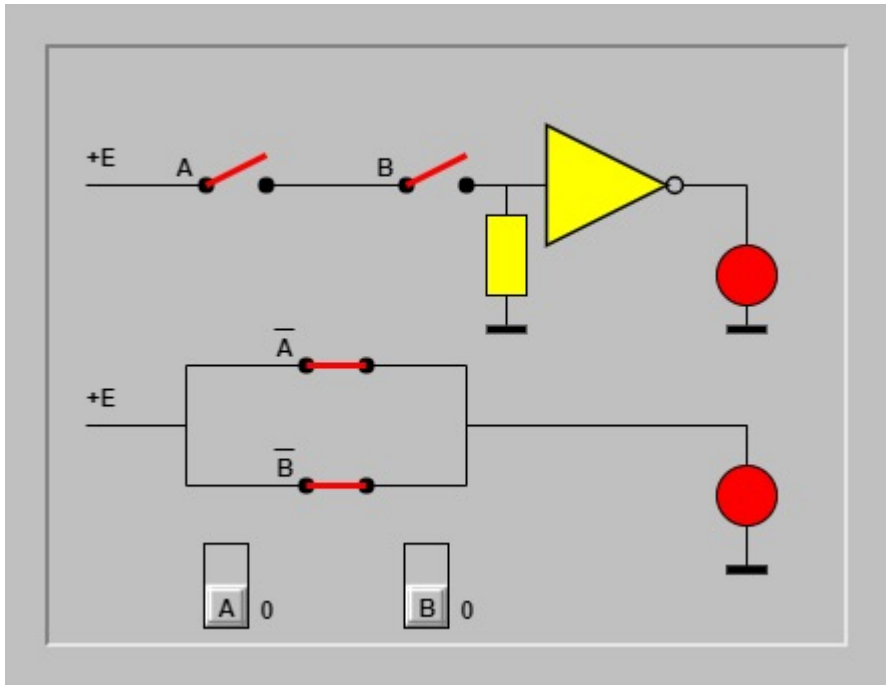
Théorème de *De Morgan*

Les variables sont représentées par des interrupteurs placés dans le circuit d'alimentation d'une lampe témoin.





Ces deux théorèmes sont très utilisés pour simplifier des propositions logiques. Ils sont généralisables à n variables.



<u>Théorème 1</u> :
Le complémentaire du OU des variables est égal au ET des complémentaires de chaque variable.
non (A ou B) = (non A) et (non B).

<u>Théorème 2</u> :
Le complémentaire du ET des variables est égal au OU des complémentaires de chaque variable.
non (A et B) = (non A) ou (non B).

Conditions inverses

✓ Le tableau suivant montre des exemples de condition inverses :

Conditions inverses	
Le contraire de	**est**
`a==5`	`a!=5`
`a!=5`	`a==5`
`a > 5`	`a <= 5`
`a <= 5`	`a > 5`
`a < 5`	`a >= 5`
`a >= 5`	`a < 5`
`(a==5) && (b==8)`	`(a!=5) || (b!=8)`
`(a==5) || (b==8)`	`(a!=5) && (b!=8)`



La maîtrise de l'algèbre de Boole permet généralement de simplifier l'écriture des conditions logiques, rendant ainsi le programme bien plus lisible, et sans en changer son fonctionnement.

L'instruction switch (1/2)



- ✓ Elle permet un choix multiple en fonction de l'évaluation d'une expression;
- ✓ La syntaxe est comme suit :

```
switch (expression) {
      case exp-1 : BlocInstruction1;
                   break;
      .
      .
      .
      case exp-N : BlocInstructionN;
                   break;
      default BlocInstruction;
}
```

L'instruction switch (2/2)



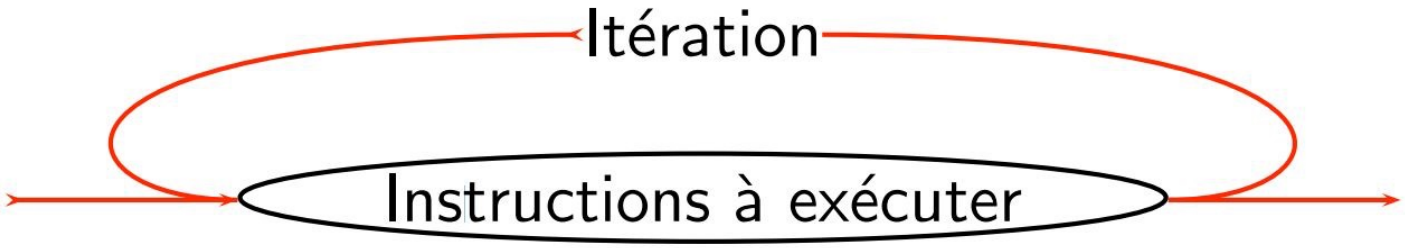
- ✓ L'instruction `break` est nécessaire pour un choix exclusif;
- ✓ L'utilisation de `default` est optionnelle;
- ✓ Tous les cas doivent être différents.

Exemple

```
public class Default {
        public static void main (String[] args) {
                int n ;
                System.out.print (« Donnez un nombre entier : ") ;
                n = Clavier.lireInt() ;
                switch (n) {
                        case 0 : System.out.println ("nul") ;
                                        break ;
                        case 1 : System.out.println ("un") ;
                                        break ;
                        default : System.out.println ("grand") ;
                }
                System.out.println ("Au revoir");
        }
}
```

Les boucles

Principe fondamental d'une boucle



✓ Itérations
- ❑ Un processus dans lequel une opération est effectuée plus d'une fois.

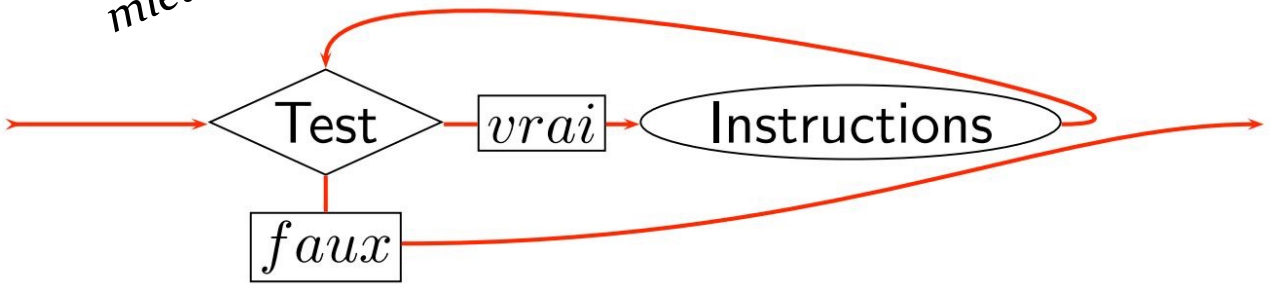
✓ Combien de fois ?
- ❑ Il faut éventuellement arrêter l'itération;
- ❑ Quand une condition devient fausse : boucle `while` (`tant que`)
- ❑ `n` fois: boucle `for` (`pour`).

✓ Il existe trois types de boucle :
- ❑ `while`
- ❑ `do ... while`
- ❑ `for`

La boucle while



Un dessin aide mieux à comprendre



✓ Syntaxe de la boucle `while` :

```
while (condition)
{
        instructions
}
```

✓ La condition est soit **vraie** soit **fausse**, et sera formulée en utilisant les opérateurs de comparaison;

✓ Le bloc d'instructions, **placé entre accolades** juste après le `while`, sera exécuté **tant que la condition est vraie**.

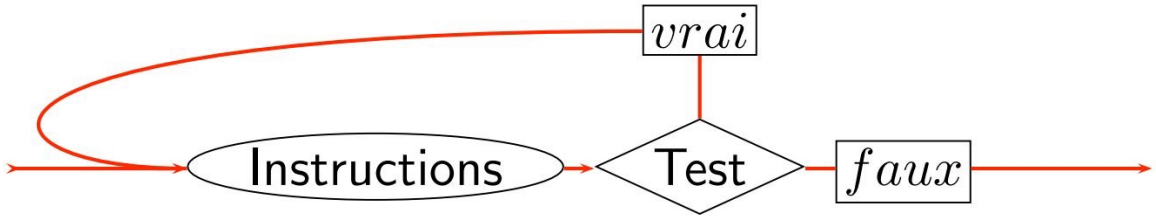
La boucle do … while



Un dessin aide mieux à comprendre

✓ Syntaxe de la boucle `do … while` :

```
do
{
        instructions
}
while (condition)
```



✓ Le fonctionnement est analogue à celui de la boucle `while` à quelques détails près :

- ❑ La condition est évaluée **après** chaque passage dans la boucle;
- ❑ On exécute le corps de la boucle **tant que la condition est vérifiée**;
- ❑ On passe toujours **au moins une fois** dans une boucle

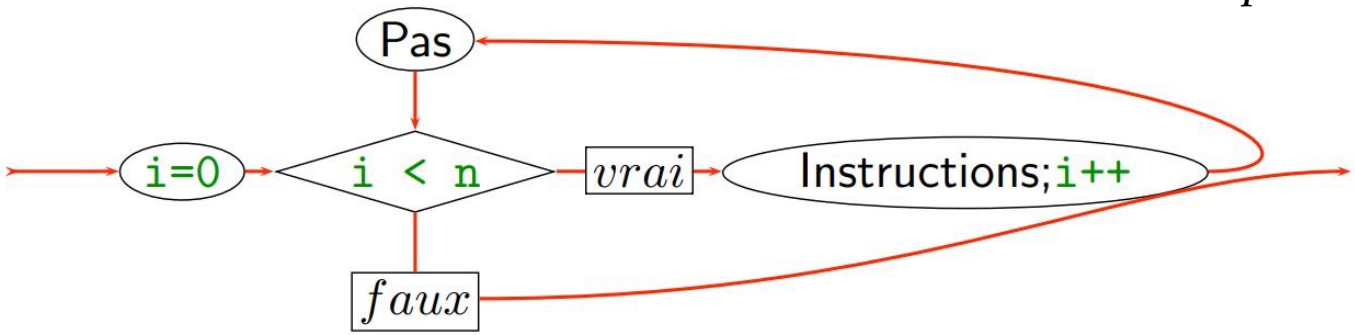
La boucle for (1/2)

✓ Syntaxe de la boucle `for` :

```
for (initialisation; condition; pas)
{
        instructions
}
```



Un dessin aide mieux à comprendre



✓ La gestion du compteur est automatique (initialisation, incrémentation, sortie de boucle);

✓ Il faut connaître à l'avance le nombre d'itérations.

La boucle for (2/2)



- ✓ En principe :
  - ❑ `for` peut contenir n 'importe quoi;

- ✓ Le plus souvent :
  - ❑ `initialisation` initialise un compteur :
    `int i = 0`
  - ❑ `itération` in/décrémente le compteur :
    `i++` ou `i -= 5`
  - ❑ `condition` vérifie si le compteur est plus grand/plus petit qu'un seuil :
    `i <= 10` ou `i > 0`

Indentation !

✓ Toujours faire attention de bien indenter en fonction des blocs;

❑ Bon exemple

```
int nb;
boolean valide = false;
do
{
    System.out.print("Entrez un nombre [1..100]: ");
    nb = Keyboard.readInt();
    if( (nb < 1) || (nb > 100) ) {
        System.err.println( "Entrée invalide." );
    }else{
        valide = true;
    }
}while( !valide );
```

❑ Mauvais exemple

```
int nb;
boolean valide = false;
do
{
System.out.print("Entrez un nombre [1..100]: ");
nb = Keyboard.readInt();
if( (nb < 1) || (nb > 100) ) {
System.err.println( "Entrée invalide." );
}else{
valide = true;
}
}while( !valide );
```

Directives pour bien indenter (1/3)



- ✓ Toute instruction dans un même bloc doit débuter à la même colonne;

- ✓ Indentation d'au moins deux espaces (par rapport au bloc englobant) :

```
englobant
{
  indenté
  ...
}
```

Directives pour bien indenter (2/3)



✓ <u>Même nombre d'espaces pour tous les blocs</u> :

```
englobant1
{
   indenté1
   ...
   englobant2
   {
      indenté2
      ...
   }
}
```

Directives pour bien indenter (3/3)



✓ Instruction coupée : la suite doit être indentée.

```
grandTotal = total + fraisBase +
  (int)((total + taxe) * TVQ + 0.5) + etc;

System.out.println( "Le cercle de rayon " +
                    r + " a un perimetre de " +
                    p );
```



C'est mieux en indentant, non ?

Questions et remerciements

« *Le pire, quand on fait un discours, ce n'est pas de prendre conscience que vos auditeurs regardent leur montre, c'est le moment où ils se mettent à la secouer pour voir si elle n'est pas arrêtée.* »

Remerciements : Un grand merci à :

Alexandre Meslé

Jean-Christophe Michel

Sébastien Roy et Balázs Kégl